

Assignment 5 - Semaphore

May 19, 2025

1 Background

In this assignment, you need to implement Semaphore in the xv6 kernel. A Semaphore is a **mutex lock with a counter**.

2 Specifications

2.1 Semaphore API

`sem_init()` initializes the semaphore at the address pointed to by `sem`. The `value` argument specifies the initial value for the semaphore.

`sem_wait()` decrements (locks) the semaphore pointed to by `sem`. If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns, immediately. If the semaphore currently has the value zero, then the call blocks until it becomes possible to perform the decrement (i.e., the semaphore value rises above zero).

`sem_post()` increments (unlocks) the semaphore pointed to by `sem`. If the semaphore's value consequently becomes greater than zero, then another process or thread blocked in a `sem_wait()` call will be woken up and proceed to lock the semaphore.

2.2 Dining Philosophers Problem

Five philosophers dine together at the same table. Each philosopher has their own plate at the table. There is a fork between each plate. The dish served is a kind of spaghetti which has to be eaten with two forks. Each philosopher can only alternately think and eat. Moreover, a philosopher can only eat their spaghetti when they have both a left and right fork. Thus two forks will only be available when their two nearest neighbors are thinking, not eating. After an individual philosopher finishes eating, they will put down both forks. The problem is how to design a concurrent algorithm such that any philosopher will not starve; i.e., each can forever continue to alternate between eating and thinking, assuming that **no philosopher can know when others may want to eat or think**.

Build commit hash:

The problem was designed to illustrate the challenges of avoiding deadlock, a system state in which no progress is possible. To see that a proper solution to this problem is not obvious, consider a proposal in which each philosopher is instructed to behave as follows:

- think unless the left fork is available; when it is, pick it up;
- think unless the right fork is available; when it is, pick it up;
- when both forks are held, eat for a fixed amount of time;
- put the left fork down;
- put the right fork down;
- repeat from the beginning.

With these instructions, the situation may arise where each philosopher holds the fork to their left; in that situation, they will all be stuck forever, waiting for the other fork to be available: it is a deadlock.

The Dining Philosophers Problem can be stated as follows: Assume there are five philosophers sitting around a circular table, doing one of two things: eating or thinking. When eating, they stop thinking, and when thinking, they stop eating. There are five bowls of spaghetti on the table, and a fork is placed between each pair of adjacent philosophers. Because it is difficult to eat spaghetti with only one fork, assume that a philosopher must use two forks to eat. They can only use the two forks on either side of their plate. The Dining Philosophers Problem is sometimes described with rice and five chopsticks instead of spaghetti and forks, as eating rice requires two chopsticks.

The problem is how to design a set of rules such that the philosophers can alternate between these two states forever without any communication, i.e., without knowing when others might want to eat or think.

This problem aims to illustrate the challenges of avoiding deadlock, a state in which a program cannot continue to run. To better understand the problem, let's assume we require the philosophers to follow these rules:

- A philosopher thinks until the left fork is available (no one else is holding it). If the left fork is available, pick it up.
- The philosopher waits for the right fork to become available. If the right fork is available, pick it up.
- If both forks have been picked up, start eating spaghetti, spending the same amount of time eating each time.
- After eating, first put down the left fork.
- Then put down the right fork.
- Start thinking (enter a loop).

This solution is flawed. When each philosopher picks up their left fork and waits for their right fork to become available, a deadlock occurs, and each philosopher will be waiting forever for the (right) other philosopher to put down their fork.

3 Hints

Please first globally search for the keywords `TODO` and `Assignment`.

Regarding the synchronization mechanisms in the xv6 kernel, you can refer to the Lab slides *Mutual Exclusion & Synchronization* and *Synchronization 2*.

You should first implement the Semaphore API in `lock.h` and `lock.c`. The former contains the definition of the `struct semaphore` structure, and the latter contains the implementation of `sem_init()`, `sem_wait()`, and `sem_post()`. Then, you should implement the Dining Philosophers Problem in `a5/checkpoint3.c`. You can find the description of the Dining Philosophers Problem in that document.

4 Checkpoints

This assignment is worth 4 points in total, with 3 Checkpoints and one report. The 3 Checkpoints are worth 1 point, 2 points, and 1 point respectively. The report does not count towards the grade but is mandatory. Your report should meet the following requirements:

- Report in PDF file format.
- Screenshots of each checkpoint running successfully.
- Record the total time spent completing this assignment.
- (Optional) Describe the difficulties you encountered while completing this assignment, or describe what guidance you think this assignment still needs.
- (Optional) You can share your thoughts on how you completed this assignment in the report.
- The more concise, the better.

Requirements You should use `make runsmp` to start **multi-core** xv6.

This assignment has no user-mode code. After the kernel starts, it will directly run the `sync_lab_main` function located in `os/a5/sync_main.c`, which will call the test functions for the 3 checkpoints respectively. If you want to test a single checkpoint, you can comment out the other function calls in this function.

Checkpoint Instructions The first Checkpoint is to initialize a Semaphore with a value of 1. Its semantics are the same as a single mutex lock.

The second Checkpoint is to test a Semaphore with a value other than 1. The third Checkpoint requires you to implement the Dining Philosophers Problem and correctly avoid deadlock. You can only use Semaphores to solve it.

Full Pass Hint If all test points are passed, you should see output similar to the following:

```
1 [INFO 0,-1] synclab_init: sync lab init
2 [INFO 0,1] checkpoint1: -> checkpoint 1 starts
3 [INFO 3,1] checkpoint1: -> checkpoint 1 passed
4 [INFO 3,1] checkpoint2: -> checkpoint 2 starts
5 [INFO 1,1] checkpoint2: -> checkpoint 2 passed
6 [INFO 1,1] checkpoint3: -> checkpoint 3 starts
7 [INFO 0,1] checkpoint3: -> checkpoint 3 passed
8 [INFO 0,1] synclab_main: all checks passed
```

5 Submission

Rename your report to report.pdf, place it in the same directory as this PDF file, and run `make handin`. This will generate a handin.zip compressed package. Upload this file to Blackboard.